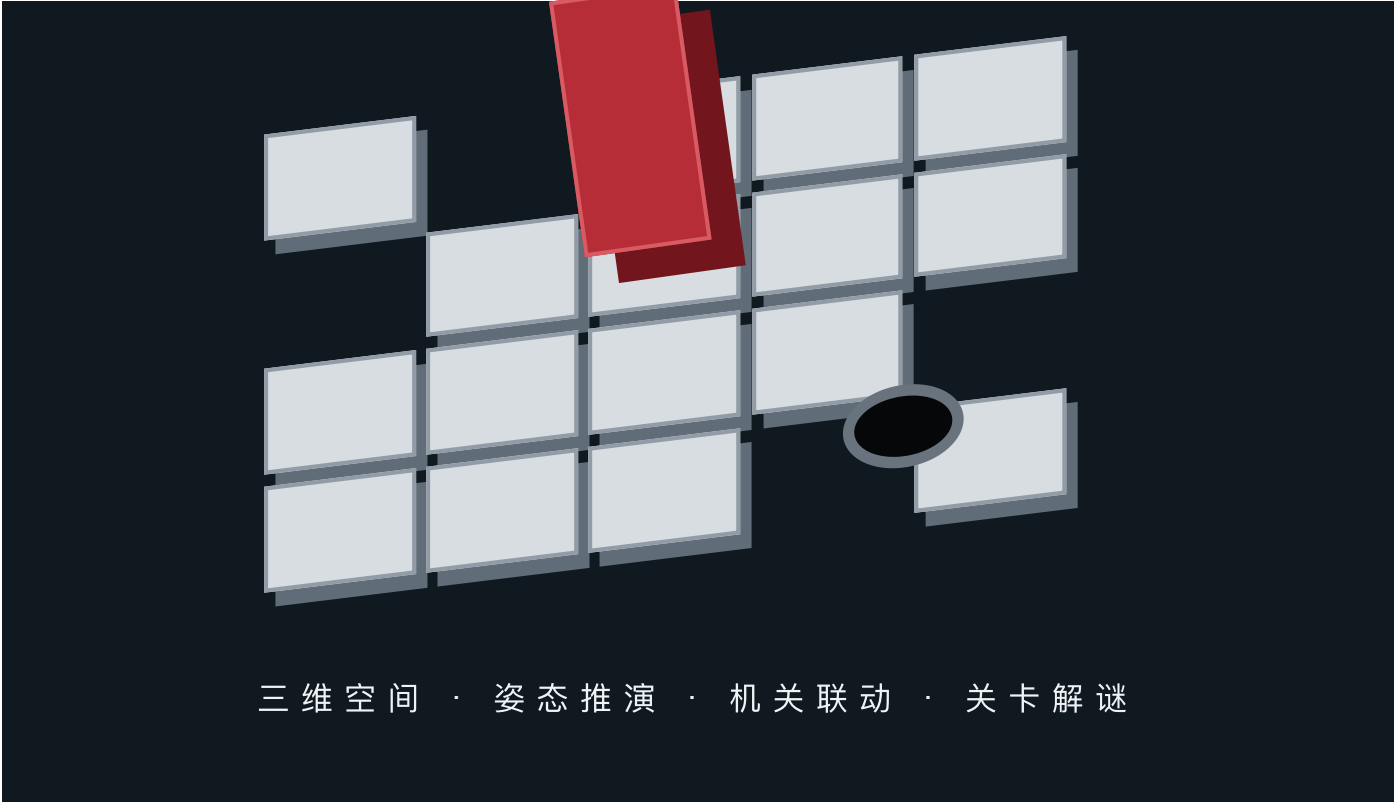


01

软件设计说明书

本文档描述滚一滚V1.0.0的总体架构、核心算法、数据结构、功能模块、接口设计及运行流程，并建立设计内容与TypeScript源程序之间的对应关系。



文档性质	软件设计说明书	实现语言	TypeScript
程序形态	移动端三维益智游戏	设计范围	客户端完整运行系统

系统以离散网格状态机为逻辑核心，以三维场景表现为交互载体。逻辑层不依赖具体渲染平台，表现层通过组件化方式组织棋盘、方块、相机、界面、音频和设备能力，从而兼顾规则正确性、运行稳定性和多端适配能力。

02

1. 系统概述与设计目标

软件面向手机和平板等横屏设备，玩家控制长方体在悬空棋盘上翻滚，通过机关改变棋盘结构，并以直立姿态落入目标洞口完成关卡。



1.1 设计目标

- 保证每次移动的姿态、占格、支撑和机关结果可确定、可复现。
 - 将规则计算与三维动画分离，动画只表现已经确认的逻辑结果。
- 支持33个关卡、关卡选择、密码直达、进度恢复和一至三星评价。
 - 支持简体中文、繁体中文和英文，并适配安全区和横屏比例。

1.2 运行与开发环境

开发环境采用Cocos Creator 3.8.8、TypeScript及Node.js 18以上版本。运行端面向具备三维图形能力的移动设备、平板设备、小游戏容器和现代浏览器；输入方式包括触屏滑动与网页键盘，画面采用正交三维相机。

03

2. 软件基本处理流程

系统从首场景装载开始，依次完成基础资源准备、运行对象创建、存档恢复和主菜单展示；进入关卡后，每次输入均经过规则计算、表现播放、状态持久化和界面更新。



2.1 启动处理

`GameplayBootstrap.onLoad()` 先设置深色清屏背景，避免启动画面结束后出现默认灰屏。字体和首页图像作为首场景依赖提前装载；若编辑器绑定缺失，则使用资源路径异步加载作为兜底。

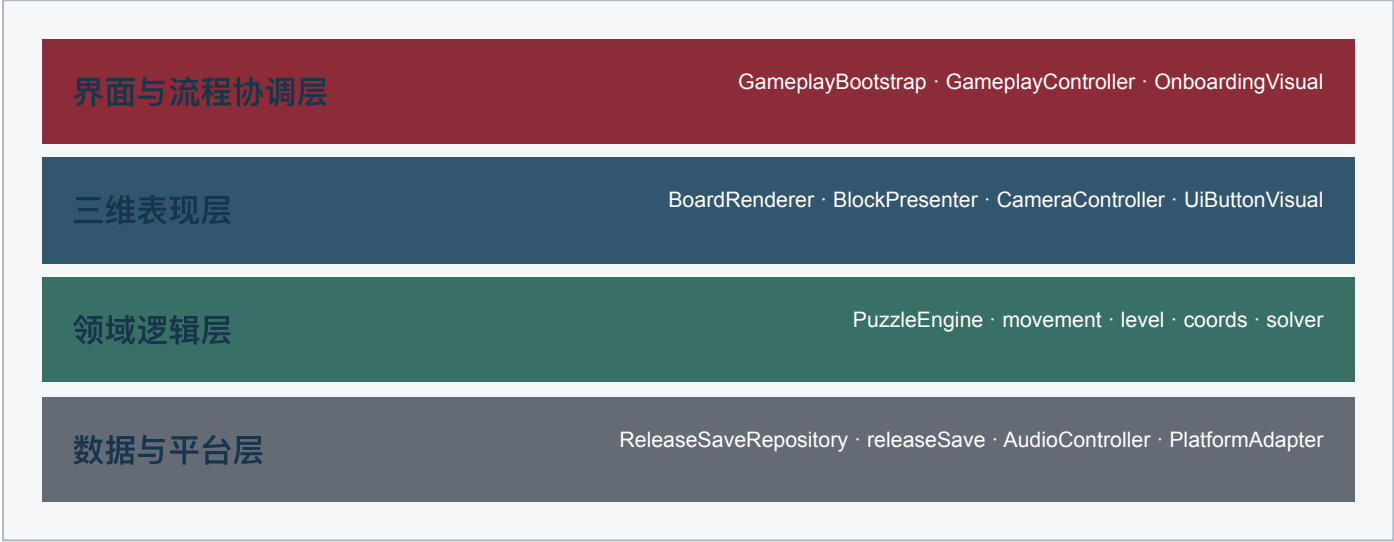
2.2 单步处理

`requestMove()` 负责输入节流和缓冲，`PuzzleEngine.move()` 产生不可变的前后状态，`BlockPresenter` 根据结果播放翻滚或掉落，完成后再由控制器处理机关表现、失败界面或通关结算。

04

3. 程序系统组织结构

程序采用“界面与流程协调层、三维表现层、领域逻辑层、数据与平台层”四层组织方式，依赖方向自上而下，核心谜题逻辑不反向依赖场景节点。



3.1 目录组织

`assets/scripts` 保存场景组件和运行控制器；`assets/scripts/shared/game` 保存可独立测试的规则与数据类型；`shared/levels` 负责关卡定义；`shared/platform` 定义设备能力抽象。

3.2 依赖原则

- 逻辑层输入为普通对象，输出为 `MoveResult`，不直接操作 `Node`或`MeshRenderer`。
- 表现层读取状态，不修改谜题规则数据。
- 控制器负责组合组件、控制模式切换和持久化时机。
- 平台差异经接口隔离，避免设备判断散落在业务代码中。

05

4. 功能模块划分

系统按照单一职责拆分为九组主要模块。模块之间通过显式属性、方法和数据对象连接，便于独立测试及后续平台移植。

模块	核心类或文件	主要职责	主要输出
启动装配	GameplayBootstrap	创建场景对象并绑定引用	完整运行图
流程控制	GameplayController	菜单、关卡、结果及模式管理	界面状态
谜题规则	PuzzleEngine	移动、机关、失败、完成与撤销	MoveResult
棋盘表现	BoardRenderer	地砖、桥梁、洞口及碎裂效果	三维棋盘
方块表现	BlockPresenter	翻滚、掉落、分裂及合并动画	方块姿态
输入系统	TouchInputController	手势解析和按钮输入隔离	Direction
存档评分	releaseSave	进度恢复、解锁和星级计算	ReleaseSaveData
适配服务	PlatformAdapter	安全区、振动和生命周期接口	设备能力
音频引导	AudioController等	音效、环境声和规则教学	反馈信息

模块划分直接对应源文件，每个组件均由启动装配模块创建，再由流程控制模块协调。规则层可在没有渲染环境的情况下运行单元测试，从而降低三维动画对规则验证的干扰。

06

5. 核心数据结构设计

领域模型使用只读TypeScript接口表达，状态更新时创建新对象，避免动画回调或界面逻辑意外修改历史状态。

LevelDefinition id / title / passcode tiles / bridges / switches / splits start / goal / solution / par	PuzzleState levelId / block / split bridgeStates / steps failed / completed
BlockState anchor: GridCoord orientation	MoveResult status / direction previous / current occupiedCells / supportedCells

5.1 坐标和姿态

`GridCoord` 采用整数 `x`、`z` 表示棋盘格。`Orientation` 包含直立、沿X轴平躺和沿Z轴平躺三种值；锚点与姿态共同确定方块占据的一格或两格。

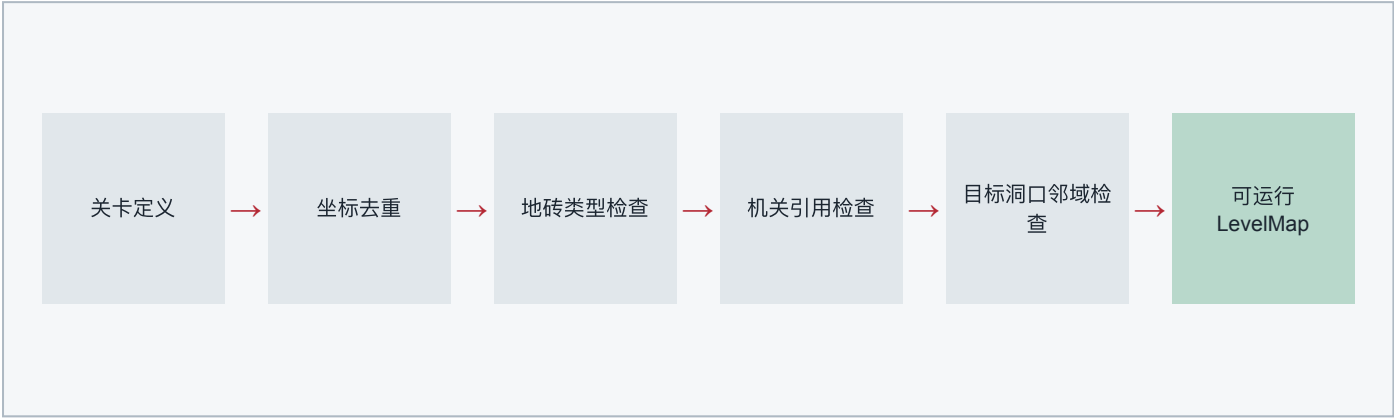
5.2 状态与结果

`PuzzleState` 是当前关卡的完整快照。`MoveResult` 同时携带移动前后状态、占格、有效支撑格和结果类型，供动画系统精确选择翻滚、悬边旋转、垂直下落或通关入洞表现。

07

6. 关卡数据与校验设计

关卡以LevelDefinition描述静态地砖、动态桥梁、机关动作、分裂目的地、起点和目标洞口。加载时先执行结构校验，错误关卡不会进入运行流程。



6.1 LevelMap索引

LevelMap 在构造时建立 `tilesByCoord`、`bridgeIdByCoord`、`switchesByCoord`、`splitsByCoord` 四类索引，将频繁的支撑和机关查询由遍历转换为键值查找。

6.2 校验规则

- 静态地砖与桥梁支撑格不得重复。
 - 起始方块占格必须落在静态地砖上。
 - 目标坐标自身不提供支撑，周边必须形成可到达结构。
- 机关动作引用的桥梁标识必须存在。
 - 分裂目的地必须不同且均有有效支撑。
 - 关卡密码使用六位数字，最优解与评分基准保持一致。

08

7. 方块翻滚算法设计

翻滚算法是纯函数rollBlock。输入当前BlockState和Direction，输出新锚点与新姿态，不读取场景，也不产生动画副作用。

当前姿态	左右移动	上下移动
standing	变为lying-x，锚点移动1或2格	变为lying-z，锚点移动1或2格
lying-x	变为standing，锚点跨过长边	保持lying-x，整体平移1格
lying-z	保持lying-z，整体平移1格	变为standing，锚点跨过长边



7.1 占格推导

occupiedCells() 根据姿态返回一个或两个坐标。所有支撑、机关和目标判断均基于该结果，而不依赖三维模型的浮点位置，因此不会受补间误差影响。

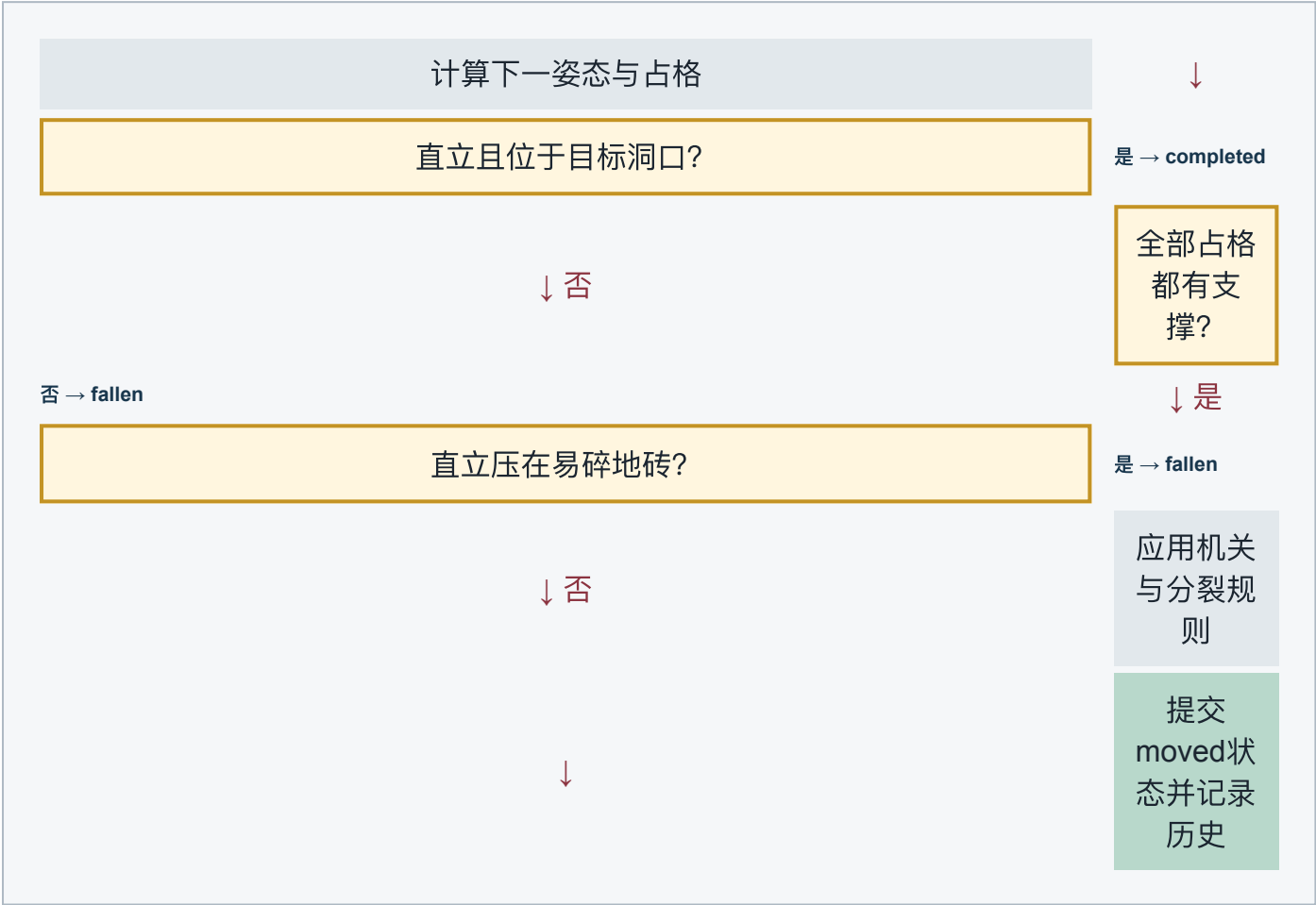
7.2 可验证性

纯函数设计便于穷举方向组合、验证姿态循环和求解器搜索。动画结束后方块节点会吸附到状态坐标，避免连续翻滚积累浮点偏差。

09

8. 移动判定与结果处理

PuzzleEngine.move()按固定优先级判断目标、支撑、易碎地砖、机关和分裂事件。相同输入状态与方向必然得到相同结果。



8.1 状态提交

`commit()` 统一写入历史栈、克隆候选状态并生成 `MoveResult`。失败状态由 `commitFailure()` 构造，非法输入则返回 `invalidResult()` 且不改变当前状态。

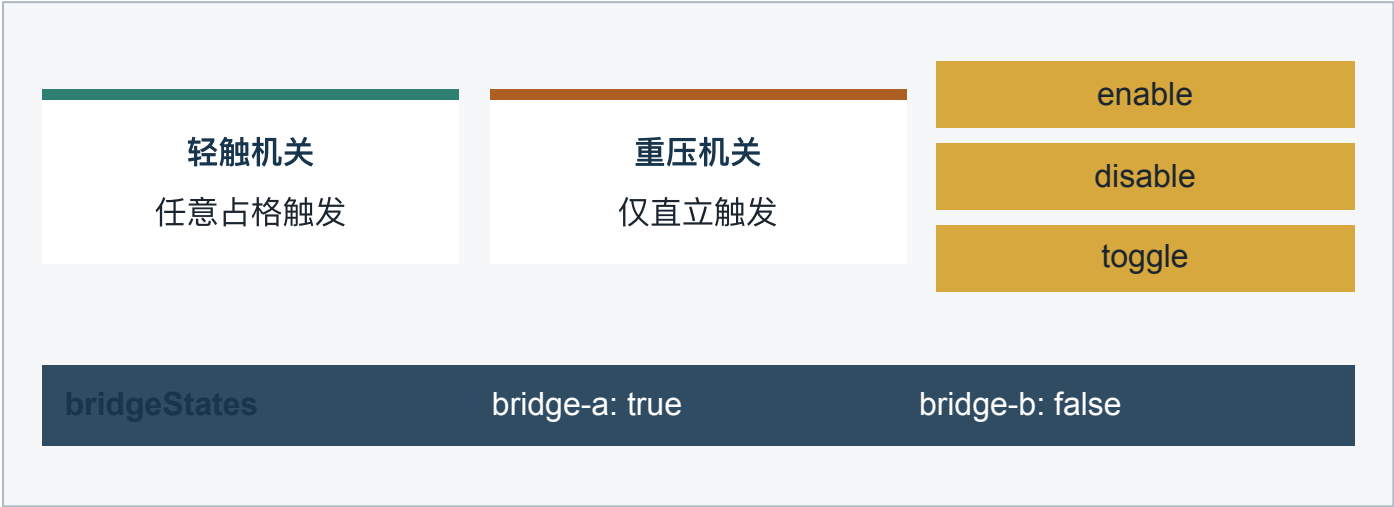
8.2 撤销和重开

每次有效操作前保存完整状态快照，`undo()` 直接恢复上一快照；`restart()` 清空历史并按照关卡起点及桥梁初始状态重新创建 `PuzzleState`。

10

9. 机关与桥梁系统设计

机关采用“触发格坐标 + 动作列表”的数据驱动方式。一个机关可控制多个桥梁，一个桥梁也可受多个机关影响。



9.1 触发规则

完整方块移动后遍历占格：轻触机关在平躺或直立时均可触发；重压机关只有方块直立时触发。分裂状态下单个小方块只能触发轻触机关。

9.2 动作语义

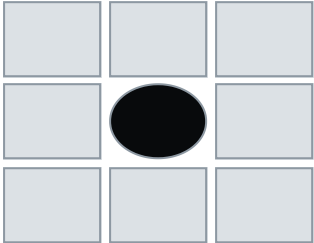
`enable` 使桥梁显示并提供支撑，`disable` 使其隐藏，`toggle` 对当前布尔值取反，因此圆形轻触机关可以重复开关桥梁。`BoardRenderer`比较前后状态并播放桥梁升降动画。

11

10. 目标洞口与易碎地砖

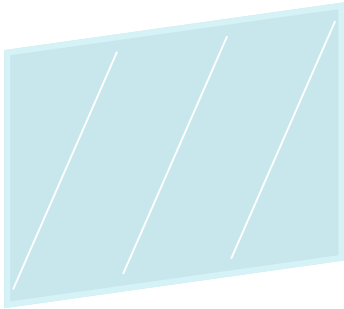
目标洞口和易碎地砖都改变普通支撑规则，但用途不同：洞口用于成功判定，易碎地砖用于限制方块直立姿态。

目标洞口



方块必须直立到达洞口坐标，随后播放入洞下落并进入通关结算。

易碎地砖



平躺经过时可支撑；直立压下时立即失败，并播放玻璃碎片散落动画。

10.1 洞口表现

`BoardRenderer.createGoalHole()` 使用井壁、底面和边框表达真实孔洞。逻辑层不把目标坐标视作普通支撑，而是在支撑判定前优先识别直立完成条件。

10.2 碎裂表现

`playFragileBreak()` 隐藏原地砖并生成九块玻璃碎片。碎片采用抛物线位移、独立旋转和末段缩放，动画结束后销毁节点，避免长期占用资源。

12

11. 分裂与重新组合机制

方块直立压上分裂地砖后，完整BlockState切换为SplitBlockState；玩家分别控制两个小方块，直到它们在相邻格重新组合。



11.1 分裂状态

`SplitBlockState` 保存两个 `GridCoord` 及 `activeCube` 索引。`switchActiveCube()` 切换活动对象并记录历史，使切换操作也能被撤销和保存。

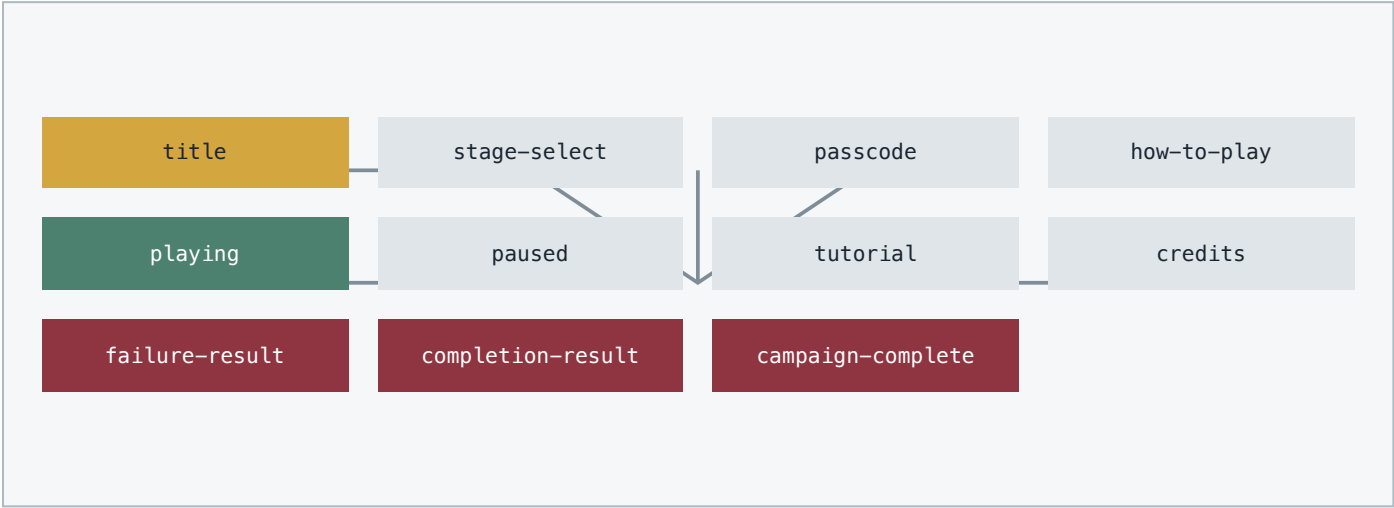
11.2 重新组合

当两个小方块同Z且X相差1时组合为 `lying-x`；同X且Z相差1时组合为 `lying-z`。除此之外保持分裂状态。`BlockPresenter`同步控制两个节点的显隐和选择标记。

13

12. 游戏流程状态机

GameplayController通过mode字段管理页面和交互权限，确保菜单输入不会误触棋盘，暂停或结果界面出现时不会继续累计时间。



12.1 模式切换

`showOnly()` 保证同一时刻仅有目标界面处于激活状态。进入playing时显示HUD并启动计时；进入paused、tutorial或结果模式时，`requestMove`拒绝新的棋盘操作。

12.2 主菜单演示

主菜单后台使用第三关的已验证路线驱动自动翻滚。演示引擎与正式游戏引擎相互独立，离开主菜单时取消补间和定时任务，避免状态泄漏。

14

13. 输入接口与操作缓冲

输入层将触屏滑动和键盘按键统一转换为Direction，再由GameplayController执行模式校验、动画忙碌检测和单步缓冲。



13.1 手势识别

`TouchInputController` 记录起点与终点，距离不足阈值时忽略；横向绝对位移较大时生成左右方向，否则生成上下方向。触点位于Button节点或其子节点时不启动手势。

13.2 操作缓冲

方块动画进行中只保留最后一次方向输入，当前动画结束后立即消费该输入，既避免并发状态更新，也使连续滑动保持流畅。网页键盘经过相机方向映射后与屏幕视觉方向一致。

15

14. 三维渲染与相机设计

表现系统按节点职责拆分。棋盘使用动态网格节点，方块使用独立旋转支点，相机采用固定斜俯视正交投影，UI由专用二维相机叠加。

Gameplay Scene

- Main Camera Orthographic / 深色清屏
- Main Light 平行光 / 阴影
- GameRoot
 - BoardRoot → BoardRenderer
 - BlockRoot → BlockPresenter
 - VoidBackdrop
 - Canvas → UI Camera

位置偏移
-6, 13, -18

俯角
约34°

偏航
约18°

投影
ORTHO

14.1 棋盘渲染

BoardRenderer按照TileDefinition创建普通、玻璃、机关和分裂地砖，并按BridgeDefinition管理动态桥梁。材质缓存以颜色和粗糙度为键，减少重复Material实例。

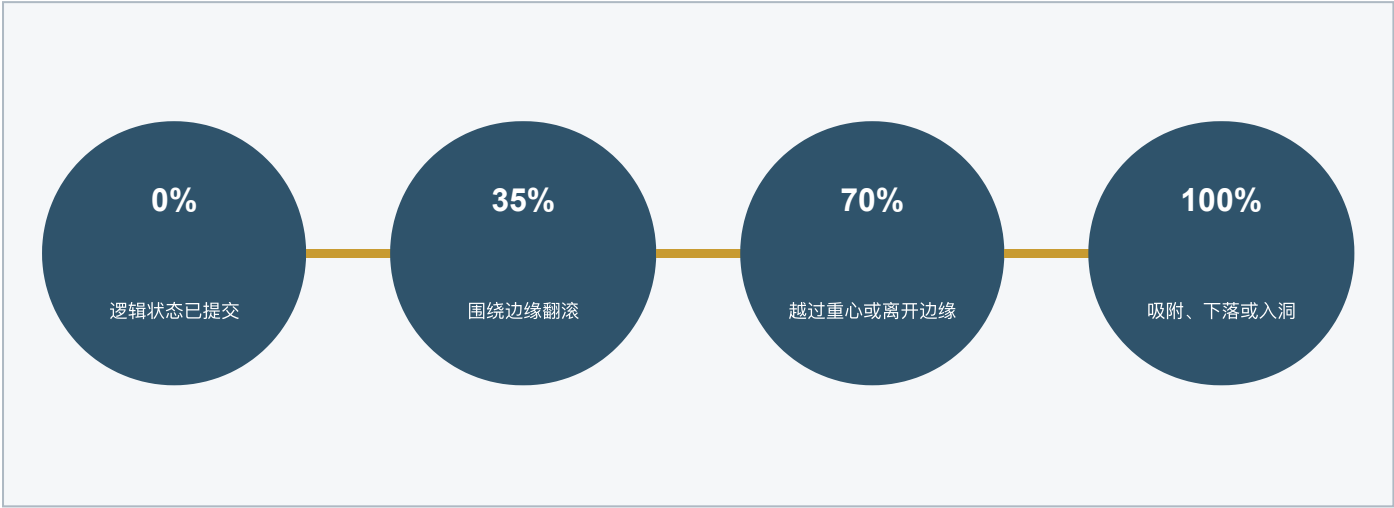
14.2 自适应取景

CameraController计算所有地砖、桥梁、分裂目的地和洞口的边界，再将三维角点投影到相机右向量和上向量，综合屏幕宽高比及菜单偏移求出orthoHeight，确保关卡完整入镜。

16

15. 方块动画与掉落设计

动画严格消费MoveResult，不参与规则判断。正常移动围绕接触边旋转；失败时根据supportedCells区分单边支撑和完全无支撑两类掉落。



存在一个支撑格

先绕棋盘边缘倾覆
再进入垂直下落

完全没有支撑格

完成越界翻滚
整个方块离开后垂直下落

15.1 翻滚支点

BlockPresenter根据方向和姿态计算旋转轴、支点及目标四元数。动画完成后调用snapTo()依据PuzzleState重建精确位置和姿态。

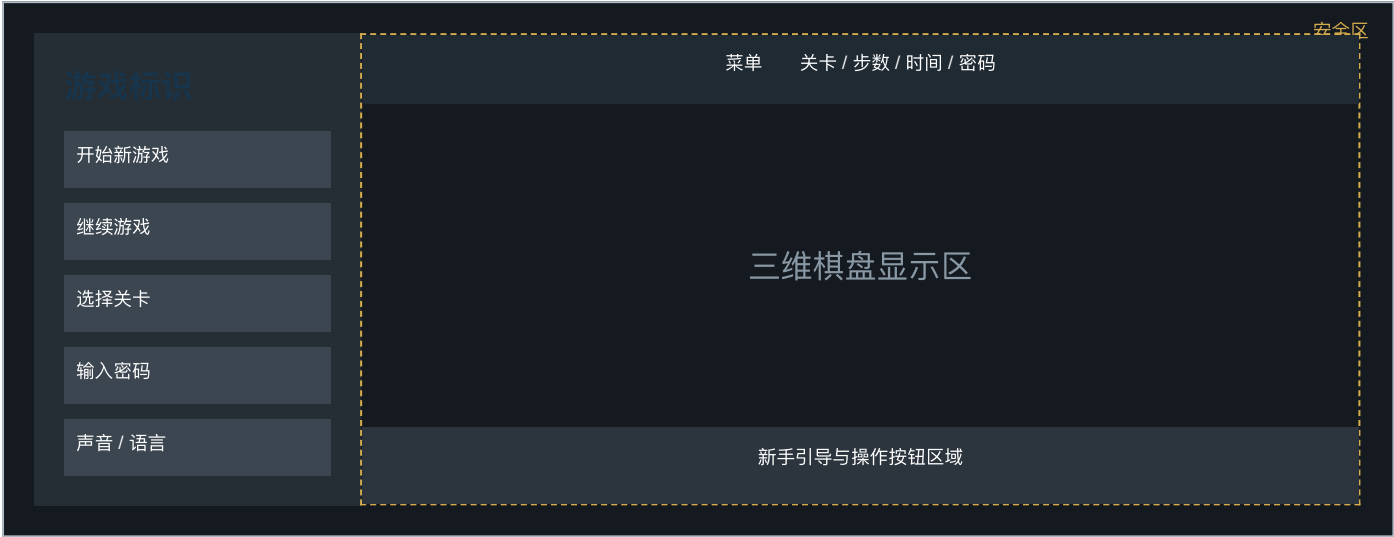
15.2 阴影稳定

方块主体保持统一受光材质，棱线采用浅灰独立几何；平行光使用单级阴影区域，避免翻滚过程中级联阴影切换导致闪烁。

17

16. 用户界面与安全区设计

界面采用横屏布局，主菜单位于左侧信息带，游戏HUD位于顶部安全区，弹窗和引导层居中或贴近底部，并由UI Camera独立渲染。



16.1 安全区

`MobileSafeArea` 读取可见区域并调整UITransform尺寸与位置，避免圆角屏、状态栏和手势区域遮挡。主菜单面板高度小于完整可见高度，并在窄屏下动态限制Logo尺寸。

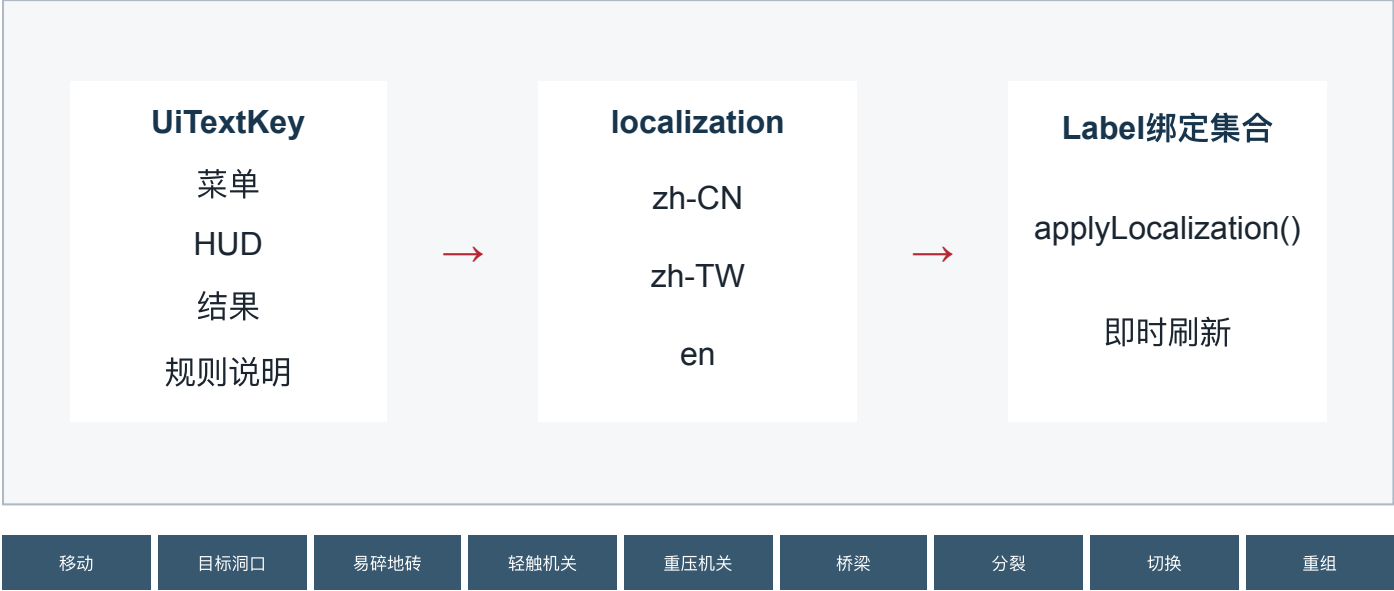
16.2 控件生成

`GameplayBootstrap`集中创建按钮、标签、面板和输入框。`UIButtonVisual`统一管理普通、悬停、按下、禁用配色；`EditBox`的提示、输入和失焦文本均保持水平及垂直居中。

18

17. 多语言与新手引导设计

文本通过UITextKey和GameLanguage索引管理，支持简体中文、繁体中文和英文。引导系统按玩家首次遇到的机制选择主题，并在存档中记录已确认主题。



17.1 引导触发

关卡加载后扫描当前LevelDefinition所包含的功能地砖，生成尚未确认的教程队列。玩家确认或跳过后调用withAcknowledgedTutorials()持久化，后续关卡不重复展示。

17.2 图形化规则

OnboardingVisual使用Graphics绘制与游戏相同的地砖、洞口、方块和机关符号。重压与分裂图标使用当前正交相机的投影基向量，确保引导角度与三维棋盘一致。

19

18. 存档、密码与评分设计

ReleaseSaveData保存继续游戏所需的最小信息。系统不直接序列化内部对象，而是保存操作序列并在恢复时重放，以验证存档仍符合当前规则。



18.1 密码访问

每关配置六位密码。输入时执行格式校验并映射关卡索引；密码列表展示全部关卡，未通关条目以星号隐藏，已通关条目可直接点击进入。

18.2 兼容与容错

存档包含版本号。解析失败、字段类型错误或关卡标识失效时回退到默认值；星级只保留合法的1至3，历史最佳成绩不会被更低评价覆盖。

20

19. 平台、音频与反馈接口

平台能力通过PlatformAdapter抽象，音频通过AudioController集中管理。业务控制器只调用统一接口，不依赖某个设备的原生实现。



19.1 音频加载

AudioController启动后异步加载环境声和常用效果，缺少单个音频时仅输出警告，不阻断游戏。玩家首次交互后启动循环声，以符合浏览器及小游戏的自动播放限制。

19.2 反馈控制

声音开关保存在ReleaseSaveData并即时更新所有标签。失败和完成时调用轻振动接口；不支持振动的平台自动忽略，不影响主流程。

21

20. 异常处理、测试与部署设计

系统通过输入校验、不可变状态、资源兜底和自动化测试控制风险。构建时仅包含项目使用的引擎模块和资源，以满足移动端及小游戏的体积要求。

验证层级	验证对象	通过标准
类型检查	TypeScript模块接口	主工程与Cocos声明均无类型错误
规则测试	移动、机关、分裂、失败、完成	状态结果与预期一致
关卡验证	33关结构和已验证解	关卡可加载且解法可完成
存档测试	恢复、重置、星级与教程记录	非法数据可降级，合法数据可重放
界面验证	横屏、安全区、三语言、输入	无越界、遮挡和方向错误
设备验证	目标移动设备与小游戏容器	启动、关卡、音频和存档稳定



20.1 异常处理

资源加载失败采用警告与降级策略；关卡结构错误在引擎创建前抛出；无效移动返回invalid而不修改状态；损坏存档回退默认结构；平台能力缺失时使用空实现。

20.2 源码关联总结

本文档中的处理流程、模块接口、数据结构和状态转换均对应现有TypeScript源程序：核心规则对应PuzzleEngine，表现对应BoardRenderer与BlockPresenter，流程对应GameplayController，装配对应GameplayBootstrap，持久化对应releaseSave与ReleaseSaveRepository。